POLITECHNIKA GDAŃSKA

Wydział ETI / ACIR

OSK

Sprawozdanie z zadania 5

Temat: Symulator mikroprocesora

Imię i nazwisko: Małgorzata Rogala, Cyprian Magnucki
Numer albumu: 198123, 197697
Grupa: A2A
Przedmiot: Organizacja systemów komputerowych
Prowadzący: dr inż. Krzysztof Cisowski

Gdańsk, 2026

Spis treści

1 Temat zadania i założenia projektowe: 2

1.1 Temat 2

1.2 Założenia projektowe 2

2 Opis przyjętych rozwiązań programowych 3

2.1 Struktura aplikacji 3

2.2 Przykładowe funkcje programu 4

3 Interfejs użytkownika 4

3.1 Realizacja operacji 5

3.2 Tryb krokowy i całościowy wykonania programu 6

3.3 Zapis i odczyt programu 6

4 Dyskusja wyników 7

4.1 Zalety aplikacji 7

4.2 Wady aplikacji 7

4.3 Możliwe kierunki rozwoju 7

1. Temat zadania i założenia projektowe:

1.1. Temat

Symulator mikroprocesora umożliwiający wykonywanie operacji przesyłania danych, dodawania i odejmowania wartości między rejestrami.

1.2. Założenia projektowe

Program miał za zadanie symulować mikroprocesor posiadający cztery 16-bitowe rejestry ogólnego przeznaczenia (AX, BX, CX, DX), w tym ich podział na część wyższą (NH) i niższą (NL), realizację dwóch trybów adresowania (rejestrowego i natychmiastowego) oraz umożliwić tworzenie prostych programów z użyciem tych operacji. Dodatkowo, program umożliwia wykonanie programu w trybie krokowym oraz całościowym, z możliwością zapisu i odczytu programu z pliku.

Do implementacji programu wybrano język Python z użyciem biblioteki Tkinter, do stworzenia interfejsu graficznego.

Aplikacja realizuje następujące założenia:

- Rejestry: Cztery 16-bitowe rejestry ogólnego przeznaczenia: AX, BX, CX, DX. Każdy z rejestrów składa się z dwóch 8-bitowych części (NH i NL).
- Rozkazy: Program obsługuje trzy podstawowe rozkazy:
  - MOV: Przesyłanie danych,
  - ADD: Dodawanie,
  - SUB: Odejmowanie.
- Tryby adresowania: Obsługiwane są dwa tryby adresowania:
  - Rejestrowy: Źródło lub cel operacji to jeden z rejestrów mikroprocesora
  - Natychmiastowy: Źródłem operacji jest bezpośrednia wartość (liczba). Programowanie: Użytkownik może tworzyć programy w formie tekstowej z instrukcjami MOV, ADD, SUB.
- Tryby wykonania:
  - Tryb całościowego wykonania: Program jest wykonywany w całości, instrukcja po instrukcji.
  - Tryb krokowy: Użytkownik ma kontrolę nad wykonywaniem programu, krok po kroku.

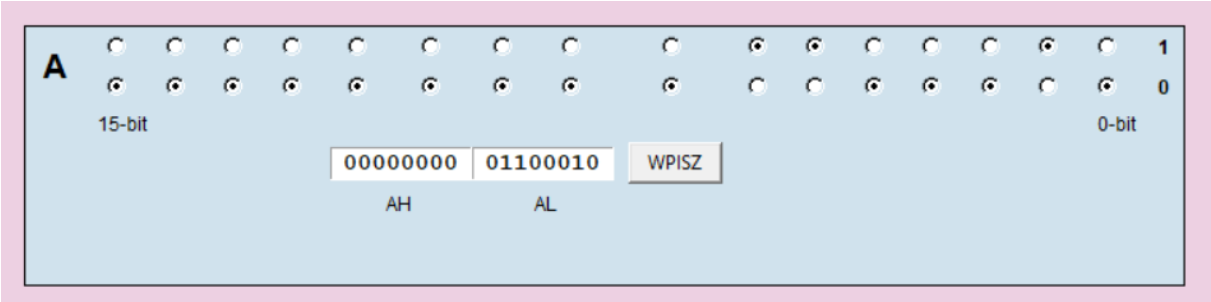
- Zapis i odczyt programu: Program umożliwia zapisanie programu do pliku oraz wczytanie wcześniej napisanego programu do edycji i ponownego uruchomienia.

2. Opis przyjętych rozwiązań programowych

2.1. Struktura aplikacji

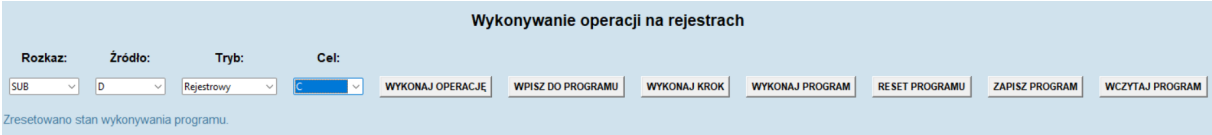
Aplikacja składa się z dwóch głównych komponentów:

- Widżet Rejestru (RejestrWidget): Każdy rejestr (AX, BX, CX, DX oraz rejestr argumentu dla trybu natychmiastowego) reprezentowany jest przez widżet, który umożliwia edycję jego wartości w postaci 16 bitów. Użytkownik może zmieniać każdy bit rejestru, a po kliknięciu przycisku "Wpisz"wartość zostaje zapisana i odzwierciedlona w rejestrze.



Rysunek 1: Widżet rejestru w GUI

- Widżet Aplikacji (Aplikacja): Aplikacja główna, która zarządza wszystkimi interakcjami, umożliwia wybór operacji (MOV, ADD, SUB), wybór źródła i celu operacji, a także trybu adresowania (rejestrowy/natychmiastowy).



Rysunek 2: Widżet aplikacji w GUI

2.2. Przykładowe funkcje programu

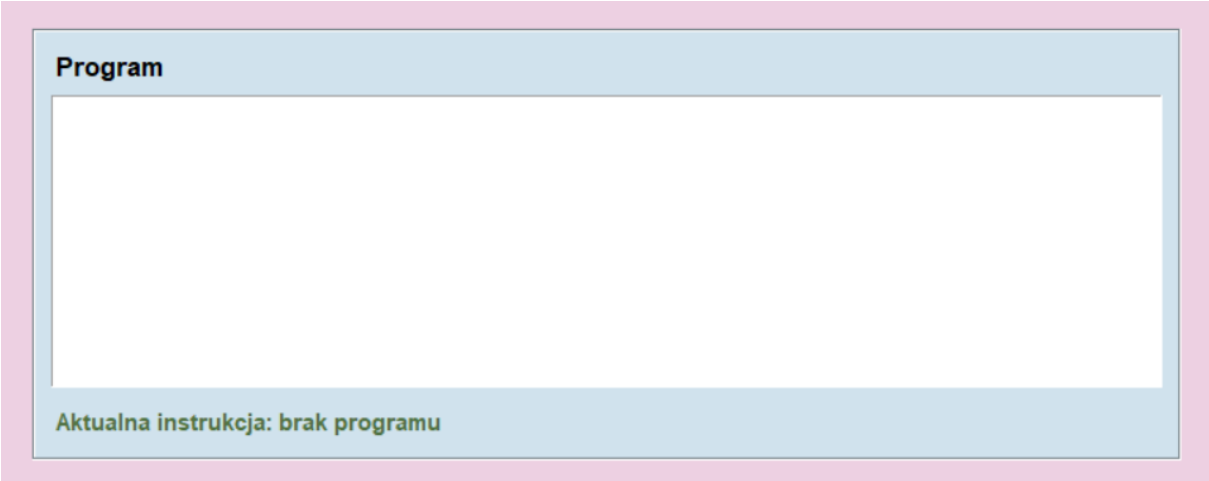
Tabela 1: Wybrane funkcje programu i ich przeznaczenie

Funkcja	**Przeznaczenie**
`__init__()`	inicjalizuje główną aplikację, ustawia tytuł okna, kolor tła i strukturę GUI
`zbuduj_gui()`	tworzy interfejs użytkownika, w tym rejestry, pola do wpisywania danych, przyciski i pole programu
`wpisz_do_rejestru()`	zapisuje wartość rejestru na podstawie ustawionych bitów w GUI
`ustaw_wartosc()`	ustawia wartość rejestru na podstawie podanej wartości, ograniczając ją do 16-bitowego zakresu
`pobierz_wartosc()`	zwraca wartość rejestru w postaci 16-bitowej
`odswiez_pola()`	aktualizuje pola AH i AL wybranego rejestru, konwertując wartość rejestru na postać binarną
`odswiez_bity()`	przypisuje wartości do każdego bitu rejestru na podstawie aktualnej wartości rejestru
`wykonaj_operacje()`	wykonuje operacje (MOV, ADD, SUB) na rejestrach na podstawie wybranych danych z interfejsu
`wpisz_do_programu()`	dodaje wybraną operację do programu w formacie assemblera

3. Interfejs użytkownika

Interfejs użytkownika składa się z kilku części:

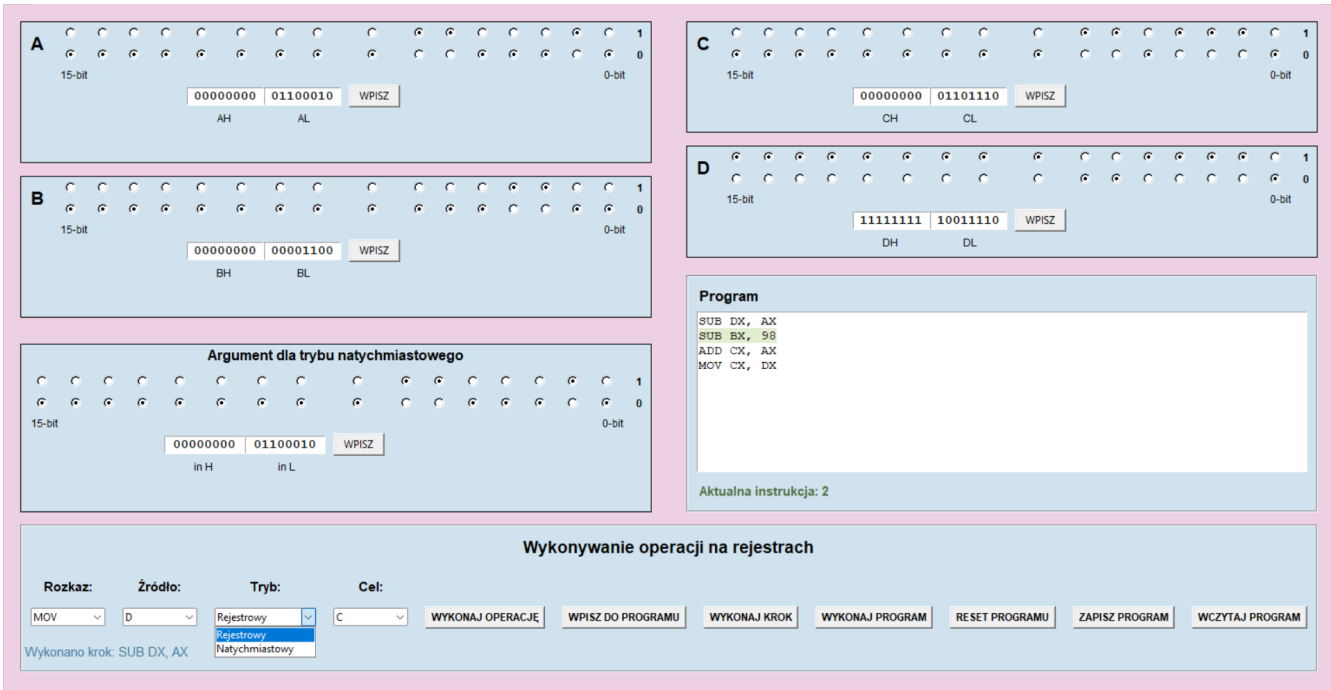
- Rejestry (A, B, C, D): Widżety przedstawiające rejestry w postaci 16-bitowych wartości, z możliwością edycji poszczególnych bitów.
- Program: Pole tekstowe, w którym użytkownik może wpisywać za pomocą przycisków instrukcje w formacie MOV, ADD, SUB. Pole to jest również wykorzystywane do wyświetlania aktualnie wykonywanych instrukcji oraz do śledzenia postępu wykonania programu.



Rysunek 3: Pole programu

- Operacje: Użytkownik może wybrać operację do wykonania na rejestrach, a także wybrać rejestr źródłowy i docelowy, a także tryb adresowania (rejestrowy lub natychmiastowy).

Wszystkie te elementy są zorganizowane w ramki i pola w taki sposób, aby użytkownik miał łatwy dostęp do wszystkich funkcji programu.



Rysunek 4: Cały interfejs programu

3.1. Realizacja operacji

Operacje takie jak MOV, ADD i SUB są realizowane za pomocą funkcji wykonujących operacje na rejestrach:

- MOV: Przypisuje wartość źródłową do rejestru docelowego.
- ADD: Dodaje wartość źródłową do rejestru docelowego.
- SUB: Odejmuje wartość źródłową od rejestru docelowego.

Operacje są wykonywane w zależności od wyboru użytkownika, który może wybrać tryb adresowania (rejestrowy lub natychmiastowy).

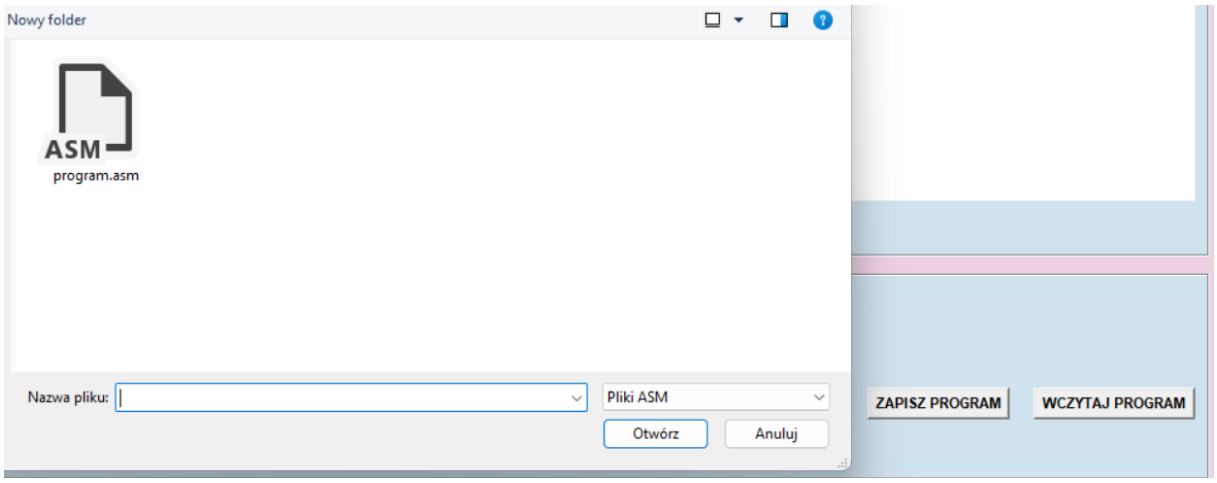
3.2. Tryb krokowy i całościowy wykonania programu

Aplikacja umożliwia wykonanie programu zarówno w trybie krokowym, jak i całościowym. W trybie krokowym użytkownik może kontrolować kolejność wykonywania instrukcji, a program informuje go, która instrukcja jest aktualnie wykonywana. W trybie całościowym program wykonuje wszystkie instrukcje po kolei.

Rysunek 5: Tryb pracy krokowej

3.3. Zapis i odczyt programu

Użytkownik ma możliwość zapisania programu do pliku (w formacie .asm lub .txt) oraz ponownego załadowania programu do aplikacji. Dzięki temu możliwe jest zapisanie sesji pracy i kontynuowanie edycji programu w przyszłości.



Rysunek 6: Zapis i odczyt z pliku

4. Dyskusja wyników

4.1. Zalety aplikacji

- interaktywny interfejs użytkownika (GUI)
- możliwość wykonywania programów w trybie krokowym i całościowym, co pozwala na naukę i testowanie instrukcji procesora w kontrolowanych warunkach
- zapis i odczyt programu z pliku tekstowego

4.2. Wady aplikacji

- brak zaawansowanych funkcji - program nie obsługuje zaawansowanych instrukcji procesora (np. operacji logicznych, skoków warunkowych).
- brak obsługi błędów w składni - przypadku błędnie wpisanej instrukcji użytkownik nie otrzymuje pełnej informacji o błędzie (np. brak obsługi błędów w przypadku błędnej składni instrukcji wczytanej z pliku).

4.3. Możliwe kierunki rozwoju

- dodanie nowych rozkazów
- rozbudowa trybów adresowania: np. adresowanie pośrednie, z przesunięciem
- zwiększenie kompatybilności z innymi językami - kompatybilność z innymi językami ożliwość pisania programów np. w języku C lub nawet wysokiego poziomu (np. Python)